Lesson 7.3 — The Interview Narrative

Lesson 7.3 — The Interview Narrative

> **Building the app got you in the room. Talking about it gets you the offer.** This is the lesson that turns 1,654 lines of code into a 90-second story.

By the end of this lesson you will:

- Have a memorized **90-second walkthrough** of the project that hits architecture, decisions, and trade-offs.
- Know how to answer the **ten interview questions** that are asked about React projects 95% of the time.
- Have a tight **vibe-coding prompt library** distilled from this entire course.
- Know how to handle the dreaded *"What would you do differently?"* question.
- Be ready to share the GitHub link with confidence.

This is the closing lesson. It's the most important. The strongest engineer in the world loses the interview to a slightly-less-strong engineer who *talks better* about their work. We're going to fix that for you.

---

1. The 90-second walkthrough

When the interviewer says *"Tell me about a project you've worked on,"* you have ninety seconds before they zone out. Use them like this:

1.1. The script (memorize this shape, swap your own words)

> *"This is **Snooker Fantasy League** — a Next.js 14 app I built for an 8-team fantasy league my friends and I run. The data is the picks each team made for*

each round of the World Snooker Championship, and the app scores those picks against actual results.

Architecturally, it's a single-page app with five tabs: standings, matches, predictions, players, and analytics. The state lives in one orchestrator component at the top — `useState` for the active tab and the selected team, `useMemo` to derive scored teams from raw team data so I don't recompute on every render. Pure scoring functions in `lib/` make the whole thing trivial to unit-test.

*The interesting decisions were two: first, picks are stored as **arrays where index matches the match-index** instead of using IDs, which keeps the scoring code five lines but enforces an invariant the type system can't catch. Second, the **analytics tab pivots team-data into match-data** to answer "where did the league agree?" — that's a cleaner mental model than trying to make the chart understand teams.*

I deployed it on Vercel through GitHub. Vitest covers the pure logic, Playwright covers the tab navigation, and Lighthouse comes back at 95+ across the board.

*If I were starting over I'd probably use a **discriminated union** for the agreement-data type — I went with optional fields and it works, but the union would be stricter."*

That's around 200 words, ~75 seconds at a normal pace. Practice it out loud until it feels conversational.

1.2. Why this script works

It hits the **five things every interviewer wants to hear**:

1. **What is it?** (Snooker Fantasy League, fantasy sports app.)
2. **What technology?** (Next.js 14, TypeScript.)
3. **What's the architecture?** (Single orchestrator, useState + useMemo, pure functions.)
4. **What was hard / what did I think about?** (Index-based picks, pivot pattern.)
5. **How is it run?** (Vercel via GitHub, tested, audited.)

And it **invites the next question** with the closing *"if I were starting over…"* line. Interviewers love when you bring up your own trade-offs — it signals you've thought past *"it works on my machine."*

1.3. The variant for non-technical interviewers

If you're talking to a recruiter or a non-engineer hiring manager:

> *"It's a fantasy sports site for an 8-team snooker league. You log in, see who's leading, drill into one team's picks, and look at charts of who agreed with whom. It's hosted free on Vercel. Took me about two weekends. I built it because I wanted a hands-on excuse to get fluent in Next.js and TypeScript, and a real spreadsheet of friends' picks was the perfect data set."*

No jargon. Outcome-first. Suitable for the first 5 minutes of any interview.

---

2. The ten questions you will be asked

Every React/Next.js interview asks variations of these. We're going to answer each one *using this app as the example*. Memorize the structure.

2.1. *"Walk me through your folder structure."*

> "Top-level: `app/` for routes, `components/` for UI, `lib/` for data and pure logic, `course/` for documentation. Inside `components/`, I split by *role*, not by *type*: a `tabs/` folder for top-level tab components, a `standings/` folder for everything that's only used inside the standings tab, and so on. The advantage is that when I want to delete the analytics tab, I delete one folder and don't have to chase down stragglers in a generic `widgets/` directory."

2.2. *"How does state management work in your app?"*

> "I deliberately *don't* use Redux or Zustand. The app has two pieces of shared state — the active tab and the selected team — and both live in the top-level `SnookerFantasyLeague` component as plain `useState`. Tab-local state, like which round is selected inside the matches tab, lives inside that tab. The rule I follow: lift state only when *something else needs it*. State libraries are tools for the day a `useState` tree becomes painful — and that day hasn't come for an 8-team app."

2.3. *"How do you decide when to extract a component?"*

"Two triggers. First, when I copy-paste the same JSX twice — the second time I write a function. Second, when a piece of JSX has a *name*: 'this is a stat card,' 'this is a path step.' Naming forces extraction. I avoid premature extraction because over-componentized React is just as painful as under-componentized — you end up chasing props through ten files instead of two."

2.4. *"How do you handle data fetching?"*

"Right now this app's data is hardcoded — eight teams, ~60 matches, all in `lib/`. So technically there's no fetching. The way I'd add it: convert `app/page.tsx` to an `async` server component, `await fetch` from a sports-results API at request time, and pass the data down. The pure scoring functions in `lib/scoring.ts` would not change at all. That's the value of separating data from logic from UI."

2.5. *"What's the trade-off between server and client components?"*

"Server components run only on the server. They can read environment variables and talk to databases, but they can't use `useState` or event handlers. Client components run in the browser. They have hooks and interactivity, but they bloat the JS bundle. The default in App Router is server, and I push `'use client'` down to leaves — only the orchestrator and any tab that uses `useState` get that directive. The static parts above stay server-rendered."

2.6. *"How do you optimize performance in React?"*

"Three layers. First, the *render layer* — `useMemo` for derived data, `React.memo` for expensive components, but only after profiling says I need them. Second, the *bundle layer* — `next/dynamic` for code-splitting heavy tabs like analytics with Recharts. Third, the *paint layer* — `loading.tsx` for instant skeletons during transitions. I never optimize before measuring; the React DevTools Profiler is my first stop."

2.7. *"How do you handle errors?"*

> "An error boundary at the root catches anything thrown during render and shows a friendly fallback. Async errors don't bubble through error boundaries, so I wrap fetches in `try/catch` and surface the message in the UI. In production I'd ship `componentDidCatch` output to Sentry. And I lean hard on TypeScript to make 'this object might be undefined' a compile-time error rather than a runtime one."

2.8. *"What's your testing strategy?"*

> "Five tests for this app. Three Vitest unit tests on the pure scoring functions — those are the highest ROI tests in the codebase. One React Testing Library integration test that renders the standings tab and asserts every team name is in the DOM. One Playwright E2E that confirms tab switching works. Plus the full TypeScript build, which is itself a giant static test. I aim for confidence, not coverage percentage."

2.9. *"How would you scale this to 1,000 teams?"*

> "Three things change. First, the data moves out of `lib/` into a database — Postgres for relational picks-vs-matches, with a real foreign key instead of array-index correspondence. Second, the orchestrator's `useMemo` over all teams becomes a paginated server-side query — you don't render 1,000 rows in a table. Third, the Recharts visualizations either get aggregated server-side or switch to a canvas-based library because SVG can't draw 1,000 bars. None of those changes touch the UI layer's component shape — that's a deliberate result of the data/logic/UI separation."

2.10. *"What would you do differently?"*

This is the one that trips most candidates. **They say "nothing."** Wrong answer. The right answer:

> "Three things. First, I'd type the agreement-data shape as a discriminated union instead of optional fields — stricter, less room for `if (datum.correct)` mistakes. Second, I'd add a tiny custom hook called `useTabState` that wraps `useState<TabId>` plus history-pushing so the URL reflects the active tab —

> right now reloading drops you to standings. Third, I'd extract the inline styles into Tailwind once the app stabilized — I went with inline styles to move fast, and that decision is now paying interest."

Three concrete, modest, technically literate trade-offs. Not "I'd rewrite it in Rust." Not "nothing."

---

3. The vibe-coding prompt library

This entire course was built around the idea that *the prompt is the source of truth*. Here's the distilled prompt library that produced this codebase. Steal these.

3.1. The *project-shaping* prompt

> "I want to build a [DOMAIN] app. The data is [DESCRIBE THE DATA SHAPE IN ONE PARAGRAPH]. The user can [DESCRIBE 3-5 USER ACTIONS]. Don't write code yet — just confirm you understand the data and the actions, and propose a 5-tab UI. Push back on anything that's ambiguous."

This is **Lesson 0 distilled**. The "don't write code yet" line is the magic ingredient. It forces the LLM to surface ambiguities you didn't see.

3.2. The *data-modeling* prompt

> "Define TypeScript interfaces for [the entities]. Be strict — no `any`, no implicit optional fields. Make every invariant explicit with a comment. If two interfaces share fields, use composition rather than inheritance."

This is **Lesson 1 distilled**. Strict types, named invariants. Output is interfaces you can almost commit verbatim.

3.3. The *pure-logic* prompt

> "Write [N] pure functions that take [INPUTS] and return [OUTPUTS]. No side effects, no React, no state. The functions should be testable in isolation. Add JSDoc comments explaining each one."

This is **Lesson 2 distilled**. Pure functions are the parts that survive every refactor. Get them right first.

3.4. The *first-component* prompt

> "Render [the simplest possible thing] using my types from `lib/types.ts`. Use a functional component with explicit props. No useState. No useMemo. Just JSX over the data. Use inline styles for now."

Lesson 3. Smallest thing first. Inline styles to move fast. State and abstraction earn their way in later.

3.5. The *progressive-enhancement* prompt

> "Add [ONE FEATURE] to [EXISTING COMPONENT]. Don't redesign anything. Don't refactor unrelated code. If the change touches more than three files, stop and tell me which files, before changing them."

Lesson 3.3 distilled, and the most useful prompt in the library. It prevents the LLM from helpfully rewriting half your project.

3.6. The *state-and-tabs* prompt

> "Refactor [SINGLE-COMPONENT FILE] into an orchestrator pattern. The orchestrator owns `[STATE PIECES]` with `useState`. Tab components receive props and a callback. Use `useMemo` for [DERIVED VALUE]. Keep all styling identical."

Lesson 4. Names the pattern, names the state, names the derivation, locks the styling. Output is clean.

3.7. The *composition* prompt

> "Extract a [COMPONENT NAME] from [PARENT FILE]. Move only the JSX that has a single responsibility — [DESCRIBE]. Pass props for [INPUTS]. Don't introduce Context. Don't change behavior."

Lesson 5. Pinpoints the unit of extraction. Forbids accidental Context introduction. Forbids behavior drift.

3.8. The *charting* prompt

> "Generate a Recharts [CHART TYPE] from this data shape: [PASTE A SAMPLE ROW]. Bars stacked by [KEY1] and [KEY2]. Use `<ResponsiveContainer>` and the `<ChartCard>` wrapper from `components/analytics/ChartCard.tsx`. Match the existing color palette."

Lesson 6. Data shape *first*. The chart-library specifics get filled in around it.

3.9. The *production-polish* prompt

> "Audit my [COMPONENT] for: (1) ARIA roles, (2) keyboard reachability, (3) loading and empty states, (4) defensive null checks. Add them in-place without changing the visual design."

Lesson 7.1. Specific list of polish items. *In-place* is the keyword that prevents the LLM from rewriting the file.

3.10. The *testing* prompt

> "Write [3 unit + 1 integration + 1 E2E] tests. Unit tests cover [PURE FUNCTION] with [3 BOUNDARY CASES]. Integration test renders [COMPONENT] with [REAL DATA] and asserts [VISIBLE BEHAVIOR]. E2E test opens the app and [USER ACTION]. Use Vitest, RTL, Playwright."

Lesson 7.2. Counts the tests. Names the tools. Names the assertions. Output runs first try most of the time.

---

4. The pattern behind every prompt

Look at all ten prompts above. They share a structure:

1. **Verb first.** "Render", "Refactor", "Extract", "Add", "Audit", "Generate", "Write".
2. **One clear deliverable.** Not a wishlist.
3. **Constraints named.** Don't introduce X. Match Y. Use Z.
4. **Existing context referenced by file path.** `components/analytics/ChartCard.tsx`, not "you know that chart wrapper I have."
5. **Failure modes anticipated.** "If this touches more than three files, stop."

This is the **vibe-coding contract**. Whenever an LLM-generated mess shows up in your codebase, look back at the prompt — it almost always violated one of those five rules.

---

5. The hardest interview questions, rehearsed

A few real ones I've been asked. Use these to practice.

5.1. *"Show me the worst code in this project. Why is it the worst?"*

Pick one. Be specific.

> "The `buildAgreementData` function in `AnalyticsTab.tsx` is the worst piece of code in the project. It's a 40-line nested function with a ternary inside a `.filter()` callback that special-cases the final round. It works, and the test suite covers it, but readability is poor. If I had another hour I'd extract it to `lib/analytics.ts`, split the finished and pending paths into two helpers, and add a discriminated-union return type."

What this signals: **you can name your own technical debt.** That's a senior trait.

5.2. *"Why didn't you use Redux / Zustand / TanStack Query?"*

> "Because the app has two pieces of shared state and zero remote data. Redux solves a problem this app doesn't have yet. TanStack Query solves a problem this app *will* have the day I add live API data — and that's the day I'd pull it in. Premature library adoption is a cost: extra learning, extra surface area for bugs, extra bundle size. I add libraries when local code starts to suffer, not preemptively."

5.3. *"How would I know if a feature you added broke something?"*

> "The CI pipeline runs lint, type-check, unit tests, integration tests, and the production build on every PR. The Playwright E2E covers the tab-switching happy path. Vercel preview deployments give a clickable URL on every PR for visual review. Beyond that, I'd want a real user-tracking and error-tracking layer in production — Sentry for crashes, PostHog or similar for usage. None of that

is in the project yet because it's a personal app, but I'd add it on day one of a paid project."

5.4. *"What's the most React-specific bug you've fixed?"*

Tell a story. The structure is **STAR**: Situation, Task, Action, Result.

> "**Situation:** I had a list of teams that re-sorted by score on every keystroke in an unrelated search box. **Task:** figure out why the chart was re-rendering when the search-box state changed. **Action:** opened React DevTools Profiler, saw that the parent re-rendered the chart's parent because its props identity changed every render — even though the underlying data was equal. I wrapped the derived data in `useMemo`. **Result:** chart re-renders dropped from once-per-keystroke to only when scores actually changed. The lesson is reference equality matters more than value equality in React's diffing."

This is **the** interview structure. Memorize STAR.

---

6. Closing the interview strong

When the interviewer asks *"Do you have any questions for me?"* — and they will — have three ready.

6.1. About the codebase

- *"What's the one part of your codebase that everyone on the team agrees needs a rewrite, but nobody has time for?"*

This question is a goldmine. It tells you exactly where the technical debt is and how the team handles it.

6.2. About the team

- *"How do you decide what's worth a unit test vs. an integration test on this team?"*

This one signals you care about testing without forcing them to ask.

6.3. About the role

- *"What does success look like in the first 90 days?"*

Forces them to articulate expectations in a way you can hold them to later.

---

7. The portfolio link

Your README is the second thing the interviewer looks at after the live URL. Make sure it has:

- A **screenshot** of the standings tab (saved as `docs/screenshot.png`).
- A **two-paragraph summary** of what the app does.
- A **bullet list of tech**: Next.js 14, TypeScript, Tailwind, Recharts, Vitest, Playwright.
- A **link to the deployed app** at the top, in bold.
- A **link to the course folder** with the line *"I wrote a 7-chapter course teaching React from scratch using this app."*

That last bullet is **massive**. It signals: *I don't just code. I teach.* That's a senior trait. Hiring committees notice.

---

8. The final vibe prompt

> "I have completed a Next.js 14 portfolio project. Help me write a 90-second elevator pitch and a `README.md` for the GitHub repo. The pitch should hit (1) what the app is, (2) the tech stack, (3) one interesting architectural decision, (4) one trade-off I'd revisit. The README should have a hero screenshot, two-paragraph summary, tech list, deployed-URL link, and a 'lessons learned' section. Tone: confident but not arrogant. Length: under 400 words for the README intro."

Everything we built in this lesson, captured in one prompt. Save it. Run it on your *next* portfolio project. The output will have the shape we just memorized.

---

9. CHECK YOURSELF

- ☐ Can you deliver the 90-second walkthrough from memory, twice in a row, without notes?
- ☐ Can you answer all ten common questions in section 2 with concrete references to *this* codebase?
- ☐ Do you have three "what would you do differently" answers ready, all modest and technically real?
- ☐ Have you memorized at least five of the ten vibe-coding prompts?
- ☐ Can you describe the difference between **server components** and **client components** without checking the docs?
- ☐ Can you tell a STAR-format story about a React bug you fixed?
- ☐ Have you prepared three questions to ask the interviewer at the end?
- ☐ Is your GitHub README in the shape described in section 7?

If yes to all eight, you are ready. Genuinely.

---

10. Where you are now

You have:

- A working Next.js 14 + TypeScript + Tailwind + Recharts app, deployed on Vercel.
- A separation of data, logic, and UI that scales beyond this project.
- Five tests covering the things that matter.
- A polished, accessible, resilient UI.
- A 90-second pitch and ten interview answers ready to roll.
- A vibe-coding prompt library you can use on the next project to move 3× faster.

You also have a 7-chapter course in your `course/` folder that **demonstrates you can teach React**, not just write it. That's the differentiator at most senior hires.

---

11. Final words

This course taught you React, Next.js, TypeScript, and a way of thinking about code that survives any framework change. The frameworks will move. The principles — **separate**

data from logic from UI, lift state only when something else needs it, write pure functions first, polish before you ship, test the things that break — those will outlast every JavaScript meta-framework that comes next.

Go to interviews. Tell the story you just rehearsed. Build the next app in a third of the prompts.

You've got this.

— *End of course.*

> Re-read `course/README.md` any time you want a refresher on the chapter map. The original single-file walkthrough lives at `course/SNOOKER_FANTASY_LEAGUE_COURSE.` for one-stop reading.